Python Programming
Unit 02 – Lecture 02 Notes
Tuples and Immutability

Tofik Ali

February 20, 2026

Contents

1 Lecture Overview **2**

2 Core Concepts **2**
- 2.1 Why Tuples? (The Idea of Immutability) . . . 2
- 2.2 Creating Tuples . . . 2
- 2.3 Indexing, Slicing, and Iteration . . . 2
- 2.4 Packing and Unpacking . . . 3
- 2.5 Tuple Methods (Small but Useful) . . . 3
- 2.6 Tuples vs Lists (How to Choose) . . . 3
- 2.7 Performance and Memory (What to Know) . . . 4

3 Common Pitfalls **4**
- 3.1 Singleton Tuples . . . 4
- 3.2 "Tuples Are Immutable" (But Inner Objects Can Mutate) . . . 4

4 Demo Walkthrough: Distance Between Two Points **4**
- 4.1 Math Behind the Demo . . . 4
- 4.2 Suggested Run Steps . . . 5

5 Quiz and Solutions **5**
- 5.1 Checkpoint 1 (Why `(5)` Is Not a Tuple) . . . 5
- 5.2 Checkpoint 2 (Unpacking with `*rest`) . . . 5
- 5.3 Checkpoint 3 (Tuple Contains a Mutable Object) . . . 5
- 5.4 Tuple or List? (Categorization) . . . 5
- 5.5 Think-Pair-Share (Where Immutability Helps) . . . 6
- 5.6 Exit Question (Convert List ↔ Tuple) . . . 6

6 Practice Exercises (With Solutions) **6**
- 6.1 Exercise 1: Use Tuples as Dictionary Keys . . . 6
- 6.2 Exercise 2: Swap and Return Multiple Values . . . 6
- 6.3 Exercise 3: Safe Update Workflow (Tuple → List → Tuple) . . . 6

7 Interview-Style Questions (Quick Revision) **7**

8 Further Reading 7

1 Lecture Overview

Tuples are Python's "fixed" (immutable) sequence type. They are used for records that should not change (coordinates, configuration values, keys, etc.). In this lecture we learn how to create tuples, access elements, unpack values, and choose tuples vs lists.

2 Core Concepts

2.1 Why Tuples? (The Idea of Immutability)

A tuple is an **immutable sequence**. "Immutable" means:

- you cannot change its length (no append/remove), and
- you cannot replace an element (no `t[0] = ...`).

Why is that useful?

- It prevents accidental changes to important values.
- It communicates intent: "this is a fixed record."
- Tuples can be used as dictionary keys *if all their elements are hashable*.

2.2 Creating Tuples

```
point = (3, 4) # parentheses
packed = 1, 2, 3 # packing without parentheses
single = (5,) # singleton tuple needs a comma
from_list = tuple([1, 2, 3])
empty = ()
```

The comma is what makes a tuple. Parentheses mostly help with readability.

2.3 Indexing, Slicing, and Iteration

Tuples support most of the *non-mutating* sequence operations:

```
t = (10, 20, 30, 40)
t[0] # 10
t[-1] # 40
t[1:3] # (20, 30)
```

You can iterate normally:

```
for value in t:
    print(value)
```

Common operations also work:

```
t = (1, 2, 2, 3)
2 in t # True (membership)
len(t) # 4
t + (9, 10) # concatenation
t * 2 # repetition
t.count(2) # 2
t.index(3) # 3
```

2.4 Packing and Unpacking

Unpacking assigns tuple positions to variables:

```
point = (10, 20)
x, y = point
```

Star-unpacking collects the remaining values into a list:

```
values = (10, 20, 30, 40, 50)
a, b, *rest = values
# a = 10, b = 20, rest = [30, 40, 50]
```

Two high-frequency patterns:

- swapping: `a, b = b, a`
- multiple return values:

```
def min_max(nums):
    return min(nums), max(nums)

mn, mx = min_max([3, 1, 7, 2])
```

2.5 Tuple Methods (Small but Useful)

Tuples have only two methods:

- `count(x)`: how many times `x` appears
- `index(x)`: position of the first occurrence of `x`

Most other operations (like sorting) are done by converting to a list or using `sorted`.

2.6 Tuples vs Lists (How to Choose)

Feature	List	Tuple
Mutability	Mutable	Immutable
Syntax	`[]`	`()`
Best use	Changeable collection	Fixed record
As dict key	No	Yes (if hashable contents)

If you expect updates (append/remove/sort in place), use a list. If the values represent a fixed record, use a tuple.

2.7 Performance and Memory (What to Know)

Because tuples are immutable, Python can store them more compactly and optimize some operations. As a result, tuples are often:

- slightly smaller in memory than lists of the same length, and
- slightly faster for iteration/access in many cases.

Important: this is a rule of thumb, not a promise. Always choose the structure that models the problem correctly (list for changeable collections, tuple for fixed records). Optimize only if you actually measure a bottleneck.

How to Measure (Optional)

You can measure memory and timing with standard libraries:

```
import sys, timeit

L = [1, 2, 3, 4, 5]
T = (1, 2, 3, 4, 5)

print(sys.getsizeof(L), sys.getsizeof(T))
print(timeit.timeit("sum(T)", globals=globals(), number=100000))
```

3 Common Pitfalls

3.1 Singleton Tuples

`(5)` is just the integer `5` in parentheses. The comma creates a singleton tuple:

```
a = (5) # int
b = (5,) # tuple
```

3.2 "Tuples Are Immutable" (But Inner Objects Can Mutate)

The tuple structure cannot change, but a tuple can contain a mutable object. Example:

```
t = ([1, 2],) # tuple containing a list
# t[0] = [9] # not allowed (would replace an element)
t[0].append(3) # allowed (mutates the inner list)
```

This surprises beginners. The rule is: *the tuple does not change; the contained object can.*

4 Demo Walkthrough: Distance Between Two Points

Script: `demo/tuple_distance.py`

4.1 Math Behind the Demo

For points $P_1 = (x_1, y_1)$ and $P_2 = (x_2, y_2)$, the Euclidean distance is:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Using tuples for points is natural because a coordinate pair is a fixed record.

4.2 Suggested Run Steps

```
python demo/tuple_distance.py
```

Suggested student activity: change input points and predict whether the distance increases or decreases before running.

5 Quiz and Solutions

5.1 Checkpoint 1 (Why `(5)` Is Not a Tuple)

Question. Why does `(5)` not create a tuple?

Solution. In Python, the **comma** makes the tuple. Parentheses are optional in many cases. `(5)` is just an integer in parentheses. To create a singleton tuple, write `(5,)`.

5.2 Checkpoint 2 (Unpacking with `*rest`)

Question. Predict the result:

```
values = (10, 20, 30, 40, 50)
a, b, *rest = values
```

What are `a`, `b`, and `rest`?

Solution. `a = 10`, `b = 20`, `rest = [30, 40, 50]`.

5.3 Checkpoint 3 (Tuple Contains a Mutable Object)

Question. Predict the output:

```
t = (1, 2, [3, 4])
t[2].append(5)
print(t)
```

Solution. The output is `(1, 2, [3, 4, 5])`. The tuple structure is unchanged, but the inner list is mutable and can be modified.

5.4 Tuple or List? (Categorization)

Prompt. For each use case, choose **tuple** or **list** and justify:

- RGB color values (fixed)
- Daily temperature readings (keeps growing)
- Coordinates on a grid (fixed record)
- Shopping cart items (user adds/removes)
- Immutable configuration (fixed)

Sample response.

- RGB values: tuple (fixed length, should not change)
- Temperature readings: list (data grows over time)

- Coordinates: tuple (fixed record)
- Shopping cart: list (add/remove items)
- Configuration: tuple (fixed, safe from accidental edits)

5.5 Think-Pair-Share (Where Immutability Helps)

Prompt. Where in programs should immutability be preferred?

Sample response. Use immutable data for values that should never change during execution (e.g., a GPS coordinate in a log entry, a configuration setting, or a composite key in a dictionary). It reduces bugs because functions cannot accidentally modify those records.

5.6 Exit Question (Convert List $\leftrightarrow$ Tuple)

Question. Write one line to convert a list `L` to a tuple and back.

Solution. `t = tuple(L)` converts list to tuple, and `L = list(t)` converts back.

6 Practice Exercises (With Solutions)

6.1 Exercise 1: Use Tuples as Dictionary Keys

Task. Store distances between cities using a tuple key `("CityA", "CityB")`.

Solution.

```
dist = {}
dist[("Dehradun", "Delhi")] = 245
print(dist[("Dehradun", "Delhi")])
```

6.2 Exercise 2: Swap and Return Multiple Values

Task. Write a function that returns both the sum and product of two numbers.

Solution.

```
def sum_and_product(a, b):
    return a + b, a * b

s, p = sum_and_product(3, 4)
```

6.3 Exercise 3: Safe Update Workflow (Tuple $\to$ List $\to$ Tuple)

Task. Given a tuple of coordinates, update the first coordinate by 1.

Solution.

```
coords = (10, 20)
tmp = list(coords)
tmp[0] += 1
coords = tuple(tmp)
```

7 Interview-Style Questions (Quick Revision)

1. **What is a tuple?** An ordered, immutable sequence type in Python.
2. **How do you create a single-element tuple?** Use a trailing comma: `(5,)`.
3. **Can a tuple contain a list?** Yes. The tuple is immutable, but the inner list can still mutate.
4. **Why use tuples as dictionary keys?** Because keys must be hashable; tuples are hashable when all elements are hashable.
5. **When should you prefer a list over a tuple?** When the collection will be modified (append/remove/sort in place).

8 Further Reading

- `https://docs.python.org/3/tutorial/datastructures.html#tuples-and-sequences`

Appendix: Slide Deck Content (Reference)

The material below is a reference copy of the slide deck content. Exercise solutions are explained in the main notes where applicable.

Title Slide

Repository: `https://github.com/tali7c/Python-Programming`

Quick Links

Core Concepts **Demo** **Interactive** **Summary**

Agenda

- Overview
- Core Concepts
- Demo
- Interactive
- Summary

Learning Outcomes

- Create tuples and use indexing and slicing
- Apply tuple packing and unpacking
- Compare tuples with lists and choose appropriately
- Use tuples in real-world scenarios

Why Tuples?

- Immutable sequences (cannot be changed)
- Useful for fixed records and safe data
- Can be used as dictionary keys (if elements are hashable)
- Communicates intent: "this should not change"

Creating Tuples

- Literal: `point = (3, 4)`
- Packing: `t = 1, 2, 3`
- Singleton requires a comma: `single = (5,)`

Packing and Unpacking

```
point = (10, 20)
x, y = point

values = (1, 2, 3, 4)
a, b, *rest = values
```

- Unpacking maps tuple positions to variables
- `*rest` collects remaining values

Worked Example: Swap and Multi-Return

```
a, b = 5, 9
a, b = b, a # swap

def min_max(nums):
    return (min(nums), max(nums))

mn, mx = min_max([3, 1, 7, 2])
```

- Tuples are a common way to return multiple values

Tuple Methods and Operations

- Methods: `count`, `index`
- Supports indexing, slicing, concatenation, repetition
- Example: `(1,2) + (3,4)`

Tuples vs Lists

Feature	List	Tuple
Mutability	Mutable	Immutable
Syntax	[]	()
Use case	Changeable data	Fixed records
Hashable	No	Yes (if items hashable)

Performance and Memory (High-level)

- Tuples are immutable, so Python can store them more compactly
- Tuples are often slightly faster to iterate than lists
- Choose by **intent**: fixed record → tuple, changing collection → list

Rule of thumb: prefer readability and correct modeling first; optimize later if needed.

Nested Tuples and Conversion

- Tuples can contain other tuples
- Convert between list and tuple as needed

```
coords = ((0, 0), (1, 1), (2, 2))
coords_list = list(coords)
coords_tuple = tuple(coords_list)
```

Pitfall: Singleton Tuples and "Immutable"

- `(5)` is an integer in parentheses; use `(5,)` for a tuple
- Tuples are immutable, but they can contain mutable objects

```
t = ([1, 2],) # tuple containing a list
t[0].append(3) # allowed: the list mutates
```

Common Use Cases

- Coordinates and geometry data
- Dictionary keys (composite keys)
- Returning multiple values from functions

Demo: Distance Between Points

- Store coordinates as tuples
- Unpack and compute distance
- Keep data immutable
- Extension: compute distances for multiple point pairs

Script: `demo/tuple_distance.py`

Checkpoint 1

Why does `(5)` not create a tuple?

1. Parentheses alone do not create tuples
2. A trailing comma is required
3. `(5)` is just an integer in parentheses

Checkpoint 2

Predict the result:

```
values = (10, 20, 30, 40, 50)
a, b, *rest = values
```

What are `a`, `b`, and `rest`?

Checkpoint 3

Predict the output:

```
t = (1, 2, [3, 4])
t[2].append(5)
print(t)
```

Tuple or List? (Quick Categorization)

For each use case, choose **tuple** or **list** and justify.

- RGB color values (fixed)
- Daily temperature readings (keeps growing)
- Coordinates on a grid (fixed record)
- Shopping cart items (user adds/removes)
- Immutable configuration (fixed)

Think-Pair-Share

Where in programs should immutability be preferred? Discuss one example from daily software use.

Summary

- Tuples are immutable sequences
- Packing/unpacking simplifies assignment and returns
- Use tuples for fixed records and keys
- Convert between lists and tuples when needed

Exit Question

Write one line to convert a list `L` to a tuple and back.